

Practical Task 3 – ETL Process for the DrinkMix Data Warehouse

Data Warehouse Technologies

BTU Cottbus–Senftenberg

1. Task Overview

This task required loading the data from the three analysed branch databases (DWT_BRANCH_1, DWT_BRANCH_2, DWT_BRANCH_4) into the star-schema data warehouse designed in PR2 (DIM_CUSTOMER, DIM_PRODUCT, DIM_BRANCH, DIM_TIME, FACT_SALES).

All logic was implemented in Oracle PL/SQL. All the code is submitted alongside this report.

Through this report we want to show what architecture we used, what challenges we faced, and how we resolved them.

2. Solution Architecture

We followed a simple staging -> match/dedup -> load pattern:

- **Staging tables** (STG_CUSTOMER, STG_PRODUCT, STG_SALES) hold the branch data once it's cleaned and normalized, but before we decide which rows belong to the same real customer or product.
- **A MATCH_KEY** on each staging row is how we recognize when two branch-local rows are actually the same customer or product.
- **Mapping tables** (CUSTOMER_BRANCH_MAP, PRODUCT_BRANCH_MAP) record which warehouse key each branch-local ID resolves to. This is the mapping the task sheet asks us to save separately, so repeated loads stay consistent and don't redo the duplicate-detection work every time.
- **The dimension and fact loads** then use these mappings to fill DIM_CUSTOMER, DIM_PRODUCT, DIM_BRANCH, DIM_TIME and FACT_SALES.

Everything is written as idempotent PL/SQL procedures (MERGE / NOT EXISTS guards), tied together by one RUN_FULL_ETL procedure, so the whole pipeline can be re-run safely whenever new branch data comes in.

3. Chronological Walkthrough

3.1 Customers – read, clean, find duplicates

Each branch stores customer data differently, so LOAD_STG_CUSTOMER reads all three into one common shape first:

- Branch 1 already has separate street/number and zip/city columns; gender codes get normalized from M/W/F.
- Branch 2 stores address as combined text ("street number", "zip city"), which we split with regular expressions. Branch 2 has no gender column at all.
- Branch 4 stores street+number combined the same way Branch 2 does; zip and city are already separate.

The task tells us a customer's address never changes across branches or time, so we use that as our duplicate check: name + street + house number + zip + city, all normalized. Two branch rows sharing that combination are the same person.

LOAD_DIM_CUSTOMER keeps one row per real customer (preferring whichever branch actually has gender data, since Branch 2 never does), and works out the age group from date of birth in the same step, using <18, 18-25, 26-35, 36-45, 46-55, 56-65, 66+.

Every branch-local ID – not just the winning one – gets written to CUSTOMER_BRANCH_MAP, so a customer's orders from any of their three branch identities can be traced back to one customer later.

3.2 Products – read, clean, match across branches

LOAD_STG_PRODUCT reads all three branches' catalogs into one shape, handling Branch 2's text-typed price and Branch 4's text-typed size, and pulling the product category from wherever each branch keeps it (a plain column in Branch 1/2, a lookup table in Branch 4).

All three branches sell the same 29 products, and – unlike customers – the branch-local product ID is actually shared across all three branches for a given product. So products are matched on that ID, not on name (see Challenge 3 below for why name-matching didn't work).

3.3 Turnover and liters

The order quantity field in every branch (AMOUNT) is a unit count, not liters and not money:

- **Liters** = quantity x bottle size, stored in FACT_SALES.QUANTITY_LITERS.
- **Turnover** = quantity x unit price, computed on read via V_FACT_SALES_TURNOVER rather than stored, since price turned out to vary by branch (Challenge 4 below) and we didn't want to add an extra column to the PR2 fact table.

3.4 Branch and time dimensions

LOAD_DIM_BRANCH seeds the three branches directly, since no source table describes the branches themselves. LOAD_DIM_TIME builds one row per calendar day, spanning the full range of order dates found across all three branches.

3.5 Putting it together

RUN_FULL_ETL calls every load procedure in the right order, so a full reload is a single call.

4. Challenges We Faced and How We Fixed Them

#	Challenge	How we fixed it
1	In the PR2 FACT_SALES table we have only used QUANTITY_LITERS – no revenue column – but the task treats turnover as its own calculation.	Left FACT_SALES as-is and computed turnover on read instead, via V_FACT_SALES_TURNOVER, by reversing the liters math (units = liters / bottle_size) and multiplying by price.

#	Challenge	How we fixed it
2	Branch-local customer IDs are independent per branch, so the same ID number in two branches can be two different people – IDs can't tell us who's a duplicate.	Matched customers on name + full address instead, per the task's own assumption that address never changes. Kept every branch-local ID in CUSTOMER_BRANCH_MAP.
3	Loading Branch 2's product price crashed with ORA-01722: invalid number. Turned out product NUM=12 has price stored as .071 – two decimal points – which a simple comma/period swap can't parse.	Wrote a CLEAN_NUMBER function that treats the <i>last</i> separator as the real decimal point and strips the rest, correctly recovering 0.71.
4	CLEAN_NUMBER then broke on plain integers like '0' with ORA-01428, because it passed 0 into REGEXP_INSTR's occurrence argument, which has to be positive.	Fixed it to check for "no separator at all" before calling REGEXP_INSTR, instead of after.
5	Matching products by name gave us 35 DIM_PRODUCT rows instead of 29. Branch 1 uses English product names ("Apple-Peach") while Branch 2/4 use German ("Apfel-Pfirsich") for the same product – so name-matching split real duplicates apart.	Confirmed the branch-local product ID is genuinely shared across all three branches, and matched on that ID instead of the name. Back to 29.
6	Even with a shared ID, the same product is priced differently per branch (e.g. 3.00 / 3.10 / 3.12). A single price per product would misprice sales from whichever branch didn't "win" the dedup.	Added UNIT_PRICE to PRODUCT_BRANCH_MAP to track what each branch actually charges. DIM_PRODUCT.UNIT_PRICE is now just an averaged reference value; turnover uses the real branch price.
8	LOAD_FACT_SALES then failed with ORA-01400: cannot insert NULL into QUANTITY_LITERS.	Traced it to Branch 2's BUYS.AMOUNT sometimes spelling the quantity out as a word ("two", "three", "four") instead of a digit. Added a PARSE_QUANTITY function that maps number words to values, falling back to CLEAN_NUMBER for anything already numeric.
9	DIM_BRANCH.GEOGRAPHICAL_REGION is NOT NULL, but none of the three branch schemas say anything about where the branches actually are.	Seeded EAST, WEST, SOUTH as a placeholder for respective branches.

5. Key Decisions and Assumptions

- Customers matched on name + full address (per the task's assumption).
- Products matched on branch-local ID, not name, since name isn't reliable across branches (translations, typos) but the ID is.
- Price is tracked per branch+product, not just per product, since it genuinely varies by branch.
- Turnover is computed on read, not stored, to avoid changing the PR2 fact table's shape.
- Age buckets: <18, 18-25, 26-35, 36-45, 46-55, 56-65, 66+.
- DIM_BRANCH.GEOGRAPHICAL_REGION is assumed to be EAST, WEST, SOUTH for branch1, branch2 and branch4 respectively.
- Every load is safe to re-run: staging gets rebuilt each time, and dimension/mapping tables only add genuinely new rows.

6. Verification

- Compared total branch rows read vs. unique customers/products found.
- Checked CUSTOMER_BRANCH_MAP / PRODUCT_BRANCH_MAP row counts against staging row counts.
- Confirmed no staged product failed to match a DIM_PRODUCT row.
- Listed every product where branches disagree on the name, just to document the translation/typo differences we found.
- Checked FACT_SALES row count, total liters, and total turnover.
- Checked the AGE_GROUP distribution.
- Compared DIM_PRODUCT's reference price against each branch's actual price to confirm branch-level pricing is captured correctly.
- Self-tested CLEAN_NUMBER and PARSE_QUANTITY against the actual bad values found in the data.

7. How to Run

I have provided each sql, Procedure or function file separately and in what order to run them in a [README.md](#) file.

8. Conclusion

All six required steps are covered: customers and products are read, cleaned and normalized from three differently-structured branch schemas; customer duplicates are detected on address and their branch mappings saved separately as required; turnover and liters are both calculated correctly, accounting for price actually varying by branch; and customers get an age group. Along the way we hit and fixed several real data issues (malformed numbers, spelled-out quantities, translated product names, and an empty branch dimension quietly blocking the fact load) – each one is documented above and commented inline in ETL_PR03.sql.